

NODES: The Agent-Native System Architecture Specification

A Complete Operational Guide and Intermediate Representation (IR) Reference for Autonomous AI Coding Agents and Human Engineers

1. Executive Summary & The Agent Alignment Problem

Modern Language Models (LLMs) and autonomous agent swarms excel at writing single functions and algorithms, but suffer from **architectural drift** when developing distributed or multi-service software. Without an explicit, persistent blueprint:

- **Context Degradation:** Models lose track of port bindings and contracts across extensive multi-turn sessions.
- **Interface Hallucination:** Downstream services assume payload schemas that mismatch upstream producer interfaces.
- **Premature Code Synthesis:** Agents jump directly from ambiguous user prompts into writing source code files without contract agreements.

nodes is a high-performance CLI tool written in Rust that provides a **Structured Intermediate Representation (IR)**. It acts as an external memory bank and contract enforcement layer. An architect agent can design the topology, define edge contracts, and assign atomic nodes to worker agents to build with zero ambiguity.

***MANDATORY AGENT HEURISTIC:** Always design and validate the topology using 'nodes' before writing code. Read contracts via '--json' to ensure 100% adherence to port numbers and data schemas.*

2. Core Architectural Primitives

The system model is composed of three atomic primitives: **Graphs**, **Nodes**, and **Edges**.

Primitive	Attributes	Semantics & Agent Guidance
Graph	name, version, nodes, edges	An isolated architectural boundary. Created via nodes --create_graph <name>, automatically establishing a standalone shell command <name> in PATH.
Node	name, tag, metadata, n_inputs, n_outputs	An atomic compute, storage, or interface unit. Ports are strictly 0-indexed: inputs span 0..(n_inputs-1) and outputs span 0..(n_outputs-1).
Edge	from_node, from_port, to_node, to_port, schema	A directed connection binding an output port to an input port. Carries an optional schema contract (e.g. OrderEvent.json or Weights.safetensors).

3. Complete Command Reference for AI Tool Invocations

Agents can call these commands directly via terminal tool execution. Every command supports standard Unix options and leading dash flexibility.

```
# 1. Graph Lifecycle
nodes --create_graph      # Generates standalone binary command
nodes graphs              # List all registered graphs in workspace
nodes delete-graph       # Delete graph and its command launcher

# 2. Node Operations (Called directly using the graph command)
--add --tag --metadata '' --ninputs --noutputs
--delete                  # Automatically cascades and removes connected edges
inspect [--json]         # Inspect node ports, incoming edges, and outgoing edges
list [--json]            # List all nodes or emit full graph IR

# 3. Connection & Wire Operations
--connect -o -i [--schema ]
--disconnect -o -i
edges [--json]           # Table or JSON array of all active connections

# 4. Topology Visualization
draw                     # Cycle-safe ASCII stage layout with pipeline depth & metrics
```

4. The Standard AI Multi-Agent Workflow

When an AI system is instructed to build a feature or architecture, it should execute the following 4-phase lifecycle:

Phase 1: Architectural Formulation (The Architect Agent)

The Architect Agent translates unstructured requirements into discrete components with defined I/O port counts and initializes the graph:

```
nodes --create_graph pdf_pipeline
pdf_pipeline --add pdf_source --tag "source" --metadata '{"format":"pdf"}' --ninputs 0 --noutputs 1
pdf_pipeline --add pdf_loader --tag "loader" --metadata '{"parser":"pdfplumber"}' --ninputs 1 --noutputs 1
pdf_pipeline --add text_chunker --tag "processor" --metadata '{"chunk_size":512}' --ninputs 1 --noutputs 1
pdf_pipeline --add vector_store --tag "sink" --metadata '{"db":"chromadb"}' --ninputs 1 --noutputs 0
```

Phase 2: Contract Binding

The Architect establishes the exact payload schemas on the directed wires before writing any code:

```
pdf_pipeline --connect pdf_source -o 0 -i 0 pdf_loader --schema "RawPdf.bytes"
pdf_pipeline --connect pdf_loader -o 0 -i 0 text_chunker --schema "ExtractedPages.json"
pdf_pipeline --connect text_chunker -o 0 -i 0 vector_store --schema "EmbeddedChunks.json"
```

Phase 3: Topology Verification

The Architect runs `draw` or `inspects list --json` to confirm zero unconnected ports, verify pipeline depth, and ensure no unintended dead ends.

Phase 4: Component Implementation (Worker Subagents)

Worker agents are dispatched in parallel. Each worker inspects its single assigned node: `pdf_pipeline inspect text_chunker --json`. The output guarantees:

- **Input Contract:** Port `in:0` receives `ExtractedPages.json`.
- **Internal Parameters:** Read from metadata (e.g. `chunk_size: 512`).
- **Output Contract:** Port `out:0` emits `EmbeddedChunks.json`.

5. Reference Case Study: High-Throughput vLLM Inference Engine

The following diagram illustrates how complex production systems with feedback loops (continuous batching and paged KV-cache allocations) are represented in nodes:



6. Strict Rules & Heuristics for AI Models

1. **Never invent port numbers:** Port indexes always start at 0. An output with `n_outputs: 2` has ports `out:0` and `out:1` only. Specifying port 2 will trigger a bounds error.
2. **Structured Metadata:** Place runtime configurations, port numbers, database names, and frameworks inside `--metadata JSON` so downstream codegen prompts can ingest them directly.
3. **Machine Parsing:** When parsing graph state programmatically, always pass `--json`. Avoid regex scraping of ASCII tables.
4. **Cycle Safety:** The draw command is fully cycle-safe. Feel free to model request-response loops or cache cycles without fear of infinite loops.